QMOOD-Based Software Quality Evolution Analysis of Logback with LLM-Supported Evaluation

Course: Software Architectures and Design Methods

Name:MD Humayun Kibria Shakib

ID: 255129036

Date: 18 June 2026

1. Introduction

Software quality does not stay fixed during the lifetime of a project. As new versions are released, the code may grow, become simpler, become more complex, or change structurally because of bug fixes and feature updates. For this reason, comparing several versions of the same software can give a useful view of how its design quality evolves over time.

In this project, Logback was analyzed across ten recent versions, from v_1.5.25 to v_1.5.34. The analysis focused only on the production source code in the logback-core and logback-classic modules. CK Tool was used to extract object-oriented metrics, and these metrics were then summarized and mapped to QMOOD-related quality attributes.

The purpose of the study is not to claim a complete industrial-level quality assessment. Instead, the project presents a transparent metric-based analysis suitable for a master's course project. Because CK Tool does not provide every original QMOOD design property directly, the calculation is treated as a QMOOD-inspired approximation. The results are therefore used to compare the selected versions with each other, not to assign absolute software quality values.

2. Literature Review

Object-oriented software quality is often studied using metrics such as coupling, cohesion, complexity, inheritance, and class size. These metrics help describe structural properties of a system without requiring subjective opinions about every class.

The QMOOD model is one known model for connecting object-oriented design properties to higher-level quality attributes. It relates design measurements to attributes such as reusability, flexibility, understandability, functionality, extendibility, and effectiveness.

CK Tool is a metric extraction tool for Java projects. It provides class-level metrics such as CBO, WMC, RFC, LCOM, DIT, NOC, LOC, method counts, and field counts. These metrics are useful for this project, but CK Tool does not directly provide every original QMOOD design property.

Large language models can also be used to interpret metric results. In this project, I used ChatGPT, Gemini, and DeepSeek manually through their chat interfaces. I did not use an API, and I did not ask the models to inspect the Logback source code directly.

3. QMOOD Model Overview

QMOOD connects lower-level object-oriented design properties to higher-level quality attributes. The original model includes design properties such as design size, coupling, cohesion, abstraction, encapsulation, composition, inheritance, polymorphism, complexity, and messaging.

In this project, the original QMOOD idea is used as a guide. The calculation is QMOOD-inspired because the available CK metrics do not cover every QMOOD property exactly. For example, CBO supports coupling analysis, WMC supports complexity analysis, RFC supports messaging analysis, and LCOM supports cohesion analysis.

The quality attributes used in this project are:

- Reusability
- Flexibility
- Understandability
- Functionality
- Extendibility
- Effectiveness

These scores should be read as comparative values across the selected versions, not as absolute software quality values.

4. Selected Software System

The selected software system is Logback, an open-source Java logging framework. The repository used for the project is:

https://github.com/qos-ch/logback

The selected versions are:

- `v_1.5.25`
- `v_1.5.26`
- `v_1.5.27`
- `v_1.5.28`

- `v_1.5.29`
- `v_1.5.30`
- `v_1.5.31`
- `v_1.5.32`
- `v_1.5.33`
- `v_1.5.34`

Only two production modules were analyzed:

- `logback-core/src/main/java`
- `logback-classic/src/main/java`

I did not analyze test folders, examples, target folders, generated files, documentation, or `logback-access`. This keeps the comparison fair across versions because the same main production modules are measured each time.

5. Methodology

The project followed these steps:

1. Clone the Logback repository.
2. Select ten recent `1.5.x` versions.
3. Use CK Tool to extract class-level metrics for `logback-core` and `logback-classic`.
4. Combine the class-level metrics into version-level summaries.
5. Map available CK metrics to QMOOD-related design properties.
6. Calculate QMOOD-inspired quality scores using simple formulas.
7. Generate graphs from the processed metrics.
8. Give the same metric tables and prompt to ChatGPT, Gemini, and DeepSeek.
9. Compare the LLM responses qualitatively.

No manual source-code inspection was performed. The conclusions are based on the metric outputs and on the LLM interpretations of those metric tables.

6. Metric Extraction Process

CK Tool was used to extract metrics from the two selected source folders for each version. The raw CK outputs were saved under:

`data/raw_metrics/<version>/`

For each version, the main class-level files used later were:

- `logback-coreclass.csv`
- `logback-classicclass.csv`

The raw CK files were kept unchanged. I saved processed outputs separately under:

`data/processed/`

The extraction process completed successfully for all ten selected versions.

7. QMOOD-Inspired Calculation Method

The project used available CK columns confirmed during metric inspection. The main columns used were:

- `cbo`
- `wmc`
- `rfc`
- `lcom`
- `dit`
- `noc`
- `loc`
- `totalMethodsQty`
- `totalFieldsQty`

The combined version-level metrics were saved in:

`data/processed/logback_combined_metrics.csv`

The QMOOD-inspired scores were saved in:

`data/processed/logback_qmood_scores.csv`

The mapping was:

- Design Size = class count
- Coupling = average CBO
- Complexity = average WMC
- Messaging = average RFC
- Cohesion = inverse of average LCOM
- Inheritance = average DIT and average NOC
- Abstraction = average DIT
- Polymorphism = average NOC and average method count
- Encapsulation = private method/field ratio if available
- Composition = field-related structure if available

This project used min-max normalization across the ten versions. For lower-is-better metrics, such as CBO, WMC, and LCOM, inverse normalized scores were used. For higher-is-better metrics, such as class count, RFC, DIT, NOC, method count, and field count, normal min-max scores were used.

The formulas were simple QMOOD-inspired formulas. They should be understood as a practical approximation for this course project, not as a full original QMOOD implementation.

8. Results

Table 1. Key Metric and QMOOD Score Values

Version	Class Count	Total LOC	Avg WMC	Avg LCOM	Overall QMOOD-Inspired Score	Main Interpretation
v_1.5.25	670	26,958	10.123881	21.523881	0.527702	Highest overall score; stronger early quality baseline
v_1.5.26	670	-			0.524030	Similar to v_1.5.25; quality remains high
v_1.5.28	-	-			0.409807	Noticeable decline in overall score
v_1.5.30	643	-			around 0.443	Partial recovery begins; functionality and flexibility improve
v_1.5.32	643	-			around 0.443	Recovery remains visible but understandability stays low
v_1.5.33	643	-			0.407950	Lowest overall score in the analyzed versions
v_1.5.34	643	26,292	10.297045	22.326594	0.413958	Final version remains lower than

						early versions

The processed metrics show that Logback's measured production code became smaller overall across the selected versions. The class count decreased from 670 in `v_1.5.25` to 643 from `v_1.5.30` onward. Total LOC also decreased from 26,958 in `v_1.5.25` to 26,292 in `v_1.5.34`.

At the same time, some average class-level values increased slightly. Average WMC increased from 10.123881 to 10.297045. Average LCOM increased from 21.523881 to 22.326594. This suggests that while the measured code became smaller in total, the remaining classes may have become slightly more complex and less cohesive on average.

The QMOOD-inspired overall quality score was highest in the earliest versions:

- `v_1.5.25`: 0.527702
- `v_1.5.26`: 0.524030

The lowest overall scores appeared in:

- `v_1.5.33`: 0.407950
- `v_1.5.34`: 0.413958

These values suggest an overall downward trend in the simplified quality score, but the trend is not perfectly linear. Because the scores are normalized only across these ten versions, they are best used for comparison inside this dataset.

The following graphs summarize the main metric and quality-score trends observed in the analyzed versions.

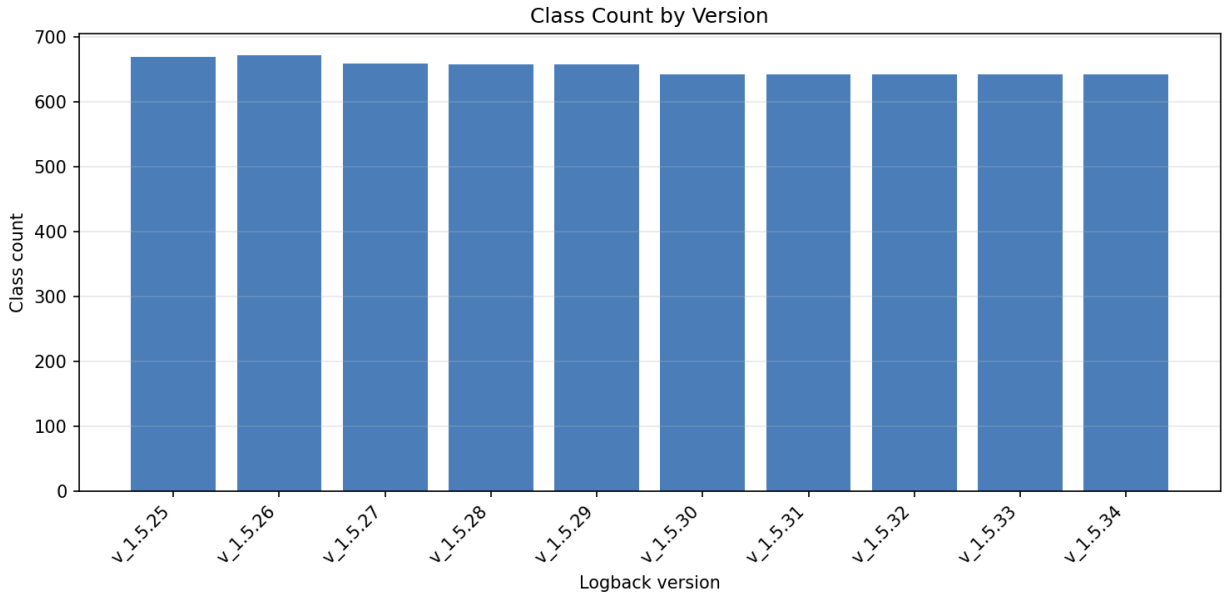

Figure 1: Class count by version

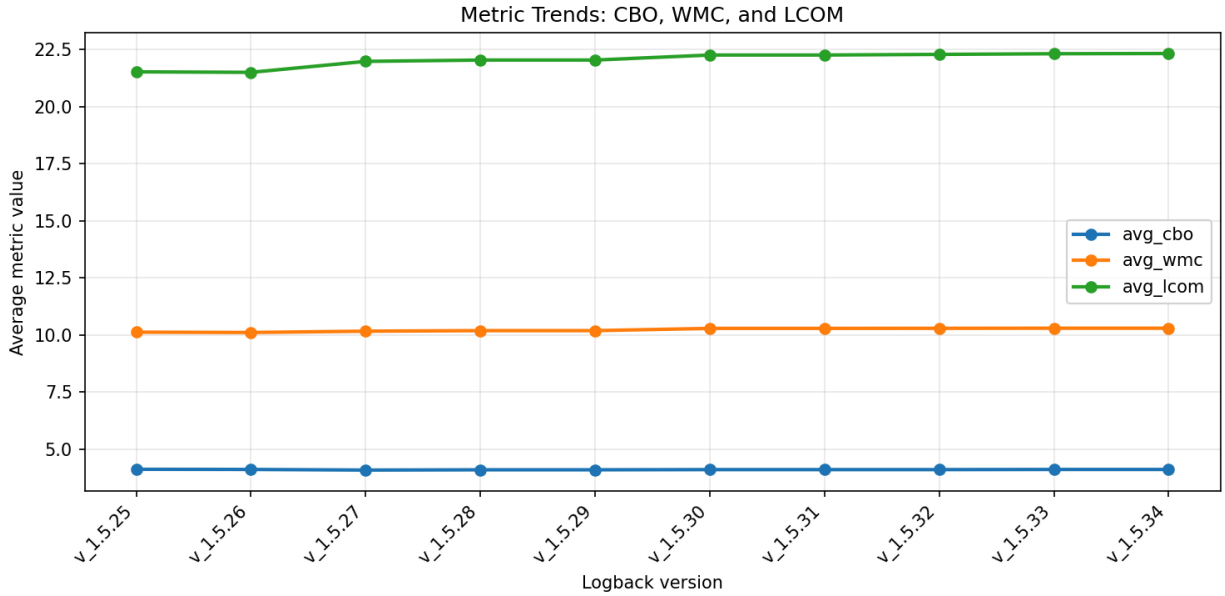

Figure 2: Metric trends for CBO, WMC, and LCOM

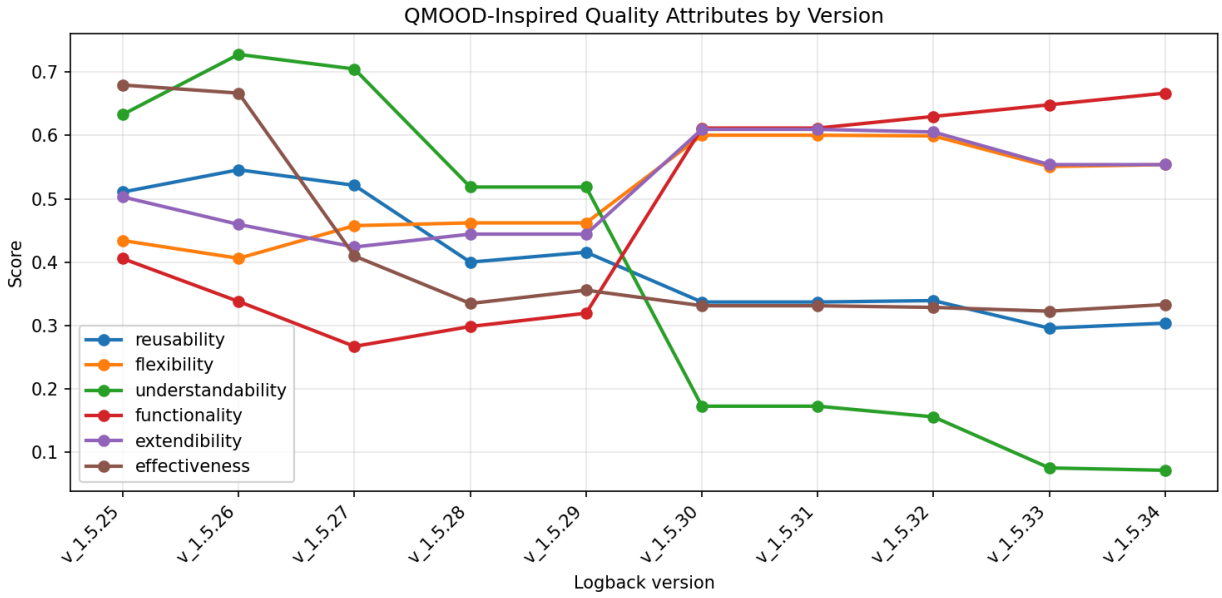

Figure 3: QMOOD quality attributes by version

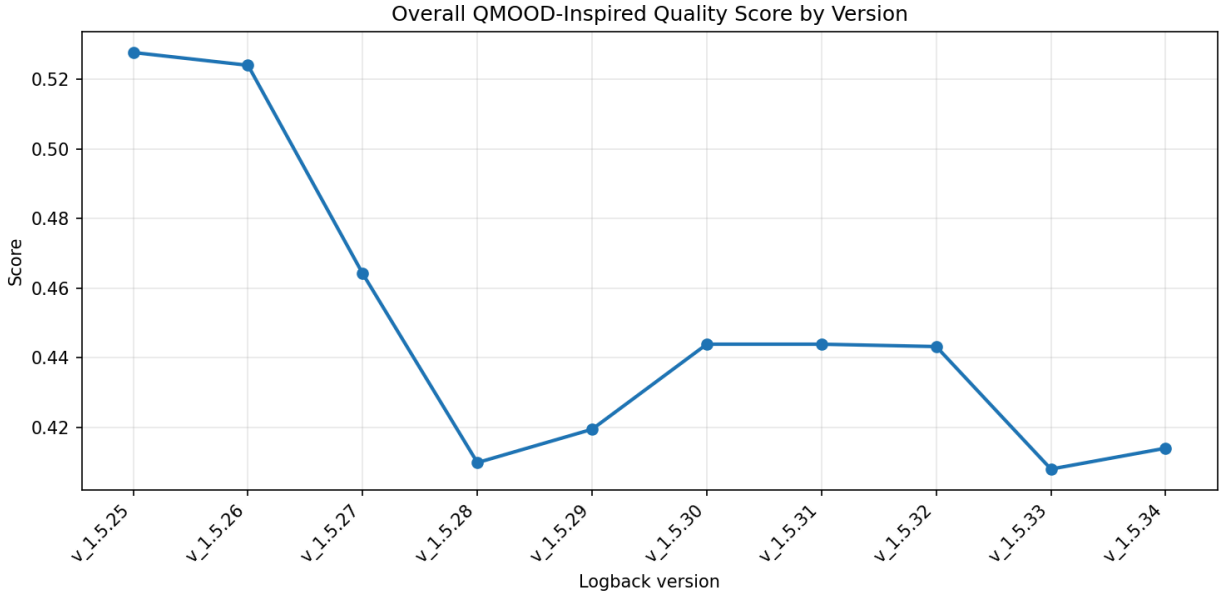

Figure 4: Overall quality score by version

9. Version-Based Quality Evolution

The quality evolution can be described in three simple phases.

First, from `v_1.5.25` to `v_1.5.28`, the overall quality score decreased from 0.527702 to 0.409807. Class count and total LOC also decreased. This may indicate code cleanup or consolidation, but the exact reason cannot be confirmed from metrics alone.

Second, from `v_1.5.29` to `v_1.5.32`, the overall score partially recovered to around 0.443. Functionality, flexibility, and extendibility improved in this period. However, understandability decreased strongly, especially after `v_1.5.30`.

Third, in `v_1.5.33` and `v_1.5.34`, the overall quality score remained relatively low. Functionality reached its highest values, but understandability and reusability stayed low compared with the earlier versions.

My main interpretation is that the later versions may show a trade-off. They appear stronger in functionality-related scores, but weaker in understandability and reusability according to this QMOOD-inspired calculation. This is still an interpretation from metrics, not proof of the developers' intent.

10. LLM-Supported Evaluation

The LLM evaluation was done manually. I gave the same prompt and the same metric tables to:

- ChatGPT
- Gemini
- DeepSeek

The prompt instructed the models not to claim that they inspected the Logback source code directly. The models were asked to use only the provided metric values.

The LLM outputs were saved in:

- `data/llm_outputs/chatgpt_response.txt`
- `data/llm_outputs/gemini_response.txt`
- `data/llm_outputs/deepseek_response.txt`

All three models identified a similar broad trend: Logback's measured quality was mostly stable in some structural areas, but later versions showed trade-offs between functionality-related improvement and maintainability-related decline. This agreement is useful, but it does not make the LLM interpretation automatically correct.

11. Comparative LLM Discussion

ChatGPT gave a balanced and cautious explanation. It described the decrease in class count, stable coupling, and the trade-off between functionality gains and lower understandability. It was useful for a simple academic explanation.

Gemini gave the most detailed metric-specific response. It used concrete values and connected them to size reduction, complexity, cohesion, flexibility, and understandability.

This made it useful for report discussion, although some parts were more interpretive than the metrics alone can fully prove.

DeepSeek gave a clear phase-based explanation. It separated the versions into decline, partial recovery, and final decline periods. This made the evolution trend easy to explain.

Overall, ChatGPT was the most cautious, Gemini was the most detailed, and DeepSeek was the clearest for phase-based trend explanation. This comparison is qualitative. I use it as supporting discussion, not as a replacement for the CK metrics or QMOOD-inspired calculations.

12. Limitations

This project has several limitations, and these limitations matter when reading the results.

First, it is not a full original QMOOD implementation. It is a QMOOD-inspired approximation based on CK Tool metrics.

Second, CK Tool does not directly provide every original QMOOD design property. Some mappings are approximate.

Third, only `logback-core` and `logback-classic` were analyzed. Other parts of the repository were excluded.

Fourth, the project used class-level metric summaries. Aggregating metrics can hide important class-level or module-level differences.

Fifth, the LLM evaluation was manual and qualitative. The models did not inspect source code. They only interpreted the provided metric tables.

Finally, the project does not explain why the metric changes happened. To answer that, future work would need changelog review, commit analysis, or manual source-code inspection.

13. Conclusion and Future Work

This project analyzed the quality evolution of Logback across ten versions using CK Tool metrics and a QMOOD-inspired scoring method. The analysis showed that the measured production modules became smaller overall, while some average class-level complexity and cohesion-related metrics became slightly worse. The overall QMOOD-inspired score declined from the earliest versions to the later versions, although there was a partial recovery around v_1.5.30 to v_1.5.32.

The LLM-supported evaluation was useful as a secondary interpretation layer. ChatGPT gave the most cautious explanation, Gemini gave the most detailed metric-based discussion, and DeepSeek gave the clearest phase-based summary. However, the LLM

outputs were treated only as supporting discussion because the models used the provided metric tables and did not inspect the source code directly.

Overall, the project shows that metric-based analysis can help identify possible software quality trends, but it cannot fully explain the reasons behind those trends. Future work could include release-note comparison, commit analysis, and selected class-level inspection to better understand why the metric changes occurred.

14. References

- Bansiya, J., and Davis, C. G. QMOOD: A Hierarchical Model for Object-Oriented Design Quality Assessment. Reference used for the QMOOD model idea.
- CK Tool by Mauricio Aniche. GitHub repository: https://github.com/mauricioaniche/ck. Accessed June 18, 2026.
- Logback project. GitHub repository: https://github.com/qos-ch/logback. Accessed June 18, 2026.
- OpenAI ChatGPT. https://chat.openai.com/. Accessed June 18, 2026.
- Google Gemini. https://gemini.google.com/. Accessed June 18, 2026.
- DeepSeek. https://www.deepseek.com/. Accessed June 18, 2026.

15. Appendix: Prompt Used for LLM Evaluation

The following prompt was used manually with ChatGPT, Gemini, and DeepSeek:

```
I am working on a master's course project named:
QMOOD-Based Software Quality Evolution Analysis of Logback with LLM-Supported
Evaluation

The analyzed software is Logback.

I analyzed only these production modules:
- logback-core
- logback-classic

The analyzed versions are v_1.5.25 to v_1.5.34.

The metrics come from CK Tool output and a simple QMOOD-inspired calculation.
These scores are approximations, not the original full QMOOD model.

Please review the metric tables I provide and give a short quality evolution
interpretation across the versions.

Important:
- Do not claim that you inspected the Logback source code directly.
- Use only the metric values I provide.
- Keep the explanation simple and suitable for a master's course project.
- Mention possible trends, improvements, or declines, but do not invent
```

missing evidence.
- If a conclusion is uncertain, say that it is uncertain.